

Backlink Agent

Azure QA Deployment Handoff

For DevOps / Platform Engineering

Product: Backlink Agent (repo: arunsaikumarB/seo-os)

Branch: master

Purpose: Deploy QA on Azure when Railway API is unavailable

Document version: 2026-08-11

IMPORTANT: Railway API trial has expired. Do not point QA at the old Railway URL. Netlify production may still reference the dead API until rebuilt against Azure.

1. Purpose of this document

This handoff explains how to deploy Backlink Agent QA on Azure, what infrastructure pieces are required, why Docker and Supabase appear in the stack, and the issues you are most likely to hit. It is written so DevOps can unblock Azure QA without reverse-engineering the monorepo.

In scope

- API container build/push to Azure Container Registry (ACR)
- API hosting on Azure Container Apps or App Service (Linux container)
- Web static hosting (Azure Static Web Apps or App Service)
- Supabase (Auth + Postgres + Storage) for QA
- Required environment variables, migrations, CORS, Auth redirects
- Smoke tests and common failure modes

Out of scope / not required for QA

- Rewriting the app to remove Supabase
- Running the full product only in Docker Compose on Azure
- Chrome Web Store publish of the Companion extension
- Production cutover / DNS for public brand domains

2. Target QA architecture

Browser

- > Azure Web (HTTPS static) apps/web Vite build -> dist/
- > Azure API (HTTPS container) apps/api/Dockerfile port 3001
 - > Supabase cloud (Auth + Postgres + Storage) [recommended for QA]
 - > Optional LLM / provider keys

Item	Detail
Web	Azure Static Web Apps OR App Service static site from apps/web/dist
API	Container Apps OR App Service (Linux) from apps/api/Dockerfile
Auth + DB	Prefer existing Supabase cloud project for QA speed
Workers	Same API container with ENABLE_WORKERS=true
Browsers	Playwright Chromium baked into API image (needs RAM/CPU)

Local developer machines mirror this: Vite on :5173, API on :3001, Supabase CLI on :54321/:54322. Local Docker is for Supabase/pgAdmin helpers - not because the whole app must run in Compose on Azure.

3. Why Docker and Supabase (FAQ for DevOps)

Why Docker locally?

Local Docker is infrastructure only. Supabase CLI starts Postgres/Auth/Storage as containers (requires Docker Desktop). Optional docker compose service provides pgAdmin. The API and web apps run with Node (npm run dev) on the host.

Why a Dockerfile for the API?

Production/QA API needs Playwright + Chromium for link probing and browser automation. The image is based on mcr.microsoft.com/playwright:v1.49.1-jammy so browser OS dependencies match. This is the artifact Azure should run - not a bare node process without browsers.

Are we still using Supabase after going local?

Yes. Local uses Supabase CLI (local stack). QA/Azure should use a cloud Supabase project. Same Auth/DB/SDK surface; only the host changes. Completely removing Supabase means rewriting Auth, data access, and Realtime - not a config

change.

What is Supabase CLI?

A command-line tool that runs a local Supabase stack and manages migrations: `supabase start` / `status` / `stop`, and `supabase db push` against a linked project.

4. Build and push API container

Build context must be the repository root (monorepo).

```
# from repo root
docker build -f apps/api/Dockerfile -t backlink-agent-api:qa .
az acr login --name <yourAcr>
docker tag backlink-agent-api:qa <yourAcr>.azurecr.io/backlink-agent-api:qa
docker push <yourAcr>.azurecr.io/backlink-agent-api:qa
```

Item	Detail
Listen port	3001 (EXPOSE 3001; app listens on 0.0.0.0)
Health	GET /health -> {"status":"ok",...}
Version	GET /v1/version
Suggested size	2 vCPU / 4 GB+ RAM (Playwright)
Startup	ensure-playwright-browsers then node apps/api/dist/index.js

Azure Container Apps / App Service: map ingress to container port 3001. Increase startup probe timeout - Chromium ensure step can take time on cold start.

5. API environment variables (Azure)

Set in Container Apps / App Service configuration. Never commit secrets.

Item	Detail
NODE_ENV	production
PORT	3001
SUPABASE_URL	QA Supabase project URL
SUPABASE_ANON_KEY	QA anon key
SUPABASE_SERVICE_ROLE_KEY	Server only - never put in web
SUPABASE_JWT_SECRET	Must match Supabase JWT secret
DATABASE_URL	Postgres URI (pooler or direct)
CORS_ORIGIN	Exact QA web origin(s), comma-separated HTTPS URLs
PROVIDER_MODE	mvp unless live providers required
ENABLE_WORKERS	true for probe / submission / browser jobs
PLAYWRIGHT_BROWSERS_PATH	/ms-playwright (image default)
PLAYWRIGHT_CHROMIUM_USE_HEADLESS_SHELL	0

Add provider keys (Gemini/OpenAI/etc.) as required by the QA org.

6. Database migrations

SQL migrations live under `supabase/migrations/` (including `109_schema_grants.sql`). Apply against the QA Supabase project before first user signup/org create.

```
npx supabase link --project-ref <qa-project-ref>
npx supabase db push
```

CRITICAL: Without `109_schema_grants.sql`, `org create` / `profile writes` fail with "permission denied for table profiles".

7. Web (static) build and deploy

Vite bakes VITE_* variables at BUILD TIME. Changing the API URL later requires a rebuild.

```
cd apps/web
# set before build:
# VITE_SUPABASE_URL=https://<qa>.supabase.co
# VITE_SUPABASE_ANON_KEY=<qa-anon>
# VITE_API_URL=https://<qa-api-host>
npm run build
# output: apps/web/dist
```

Deploy apps/web/dist to Azure Static Web Apps, Blob+CDN, or App Service static site. Ensure SPA fallback routes to index.html for client-side routing.

8. Supabase Auth URL configuration

Supabase Dashboard -> Authentication -> URL configuration:

- Site URL = QA web origin (HTTPS)
- Redirect URLs include https://<qa-web>/**
- Keep http://localhost:5173/** if developers still use local web against QA Auth

Mismatch here causes login/signup redirect failures that look like "Auth is broken" but are configuration only.

9. Challenges you may hit on Azure (and fixes)

Container OOM / Chromium crash

Playwright needs memory. Raise to 4GB+; ensure --disable-dev-shm-usage is used (image launch checks already stress Chromium). Prefer Container Apps with adequate CPU. Avoid tiny App Service plans for the API.

Cold start / health probe timeout

Startup runs ensure-playwright-browsers then boots Node. Increase health/startup probe timeout (e.g. 180s). Probe path: /health on port 3001.

CORS errors in browser

CORS_ORIGIN must list the exact QA web origin (scheme + host, no trailing slash mismatch). Rebuild is not enough - this is an API env var. Restart API after change.

Web still calls Railway / wrong API

VITE_API_URL is compile-time. Rebuild web with Azure API URL and redeploy dist. Do not expect runtime env injection for Vite without a custom shell pattern.

Auth works locally but fails on QA

Check Supabase Site URL + Redirect URLs. Confirm web and API use the SAME Supabase project keys. JWT secret on API must match that project.

permission denied for table profiles

Migrations not fully applied - especially grants migration 109. Run supabase db push against QA project.

Workers never run / Pending Analysis forever

Set ENABLE_WORKERS=true on the API container. One replica is enough for QA; multiple replicas need shared DB job semantics (pg-boss).

Image build fails / monorepo context wrong

Build from repo root with -f apps/api/Dockerfile. Do not set Docker context to apps/api only - workspace packages must be present.

ACR pull / identity errors

Grant Container Apps / App Service AcrPull on the registry. Prefer managed identity over long-lived admin passwords.

HTTPS / mixed content

Web is HTTPS; API must be HTTPS. Do not point VITE_API_URL at http:// in QA.

DATABASE_URL / pooler SSL

Use Supabase connection string appropriate for server (often pooler + sslmode=require). If /ready fails, verify network egress from Azure to Supabase is allowed.

Trying to drop Supabase for Azure Postgres only

Possible long-term, but not for this QA sprint. App Auth and data layer assume Supabase. Keep Supabase for QA; Azure hosts web + API containers.

10. Post-deploy smoke checklist

- 1. GET https://<qa-api>/health -> 200
- 2. GET https://<qa-api>/v1/version -> JSON
- 3. Open QA web -> sign up / sign in
- 4. Create organization (no profiles permission error)
- 5. Create project / run Create -> Import -> AI Review -> Generate Content -> Submit on a sample site
- 6. Confirm browser/worker jobs progress when ENABLE_WORKERS=true

11. What NOT to use for QA

- Railway API (trial expired) - do not point QA web at api-production-*.up.railway.app
- Existing Netlify prod as QA until it is rebuilt with Azure VITE_API_URL
- Committing .env, service role keys, ACR passwords, or JWT secrets
- Exposing SUPABASE_SERVICE_ROLE_KEY to the web bundle
- Combining bare docker-compose Postgres with supabase start on the same port (local only tip)

12. Companion browser extension (optional for QA demos)

Not required for Azure hosting. For demos that need Activate Package: build apps/companion, load unpacked from apps/companion/dist in Chrome. Not published to Chrome Web Store.

13. Repo pointers

Item	Detail
GitHub arunsaikumarB/seo-os (branch master)	
API Dockerfile apps/api/Dockerfile	
Web app apps/web (build -> dist)	
Migrations supabase/migrations/	
Azure notes (MD) docs/azure-qa-deploy.md	
Local setup docs/local-setup.md	
Env reference docs/environment.md	
Product name Backlink Agent (npm scopes may still say @seo-os/*)	

Questions about application behavior (AI Review, Generate Content, Companion) belong with the feature team. Questions about Azure networking, ACR, Container Apps sizing, and secrets injection belong with DevOps using this document.

14. Suggested Azure resource checklist

- Resource group for QA
- Azure Container Registry
- Container Apps Environment + API app (port 3001, env vars, AcrPull)
- Static Web App (or App Service) for web dist
- Key Vault (optional) for secrets referenced by Container Apps
- Supabase QA project + migrations applied

- DNS / custom domain later (not blocking for first QA URL)

End of handoff. Keep this PDF with the Azure runbook. Update when API port, Dockerfile base image, or required env vars change.